

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	19524281
Name	KAVIYA,G

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

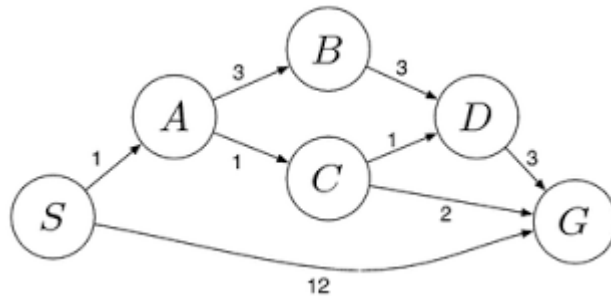
QUESTIONS: 5

Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path

from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I kaviya.G(192524281) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1: Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug?
Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available

Solve the Water Jug Problem Using Artificial Intelligence Search Strategy

Problem Statement:

You are given two water jugs:

- **Jug A = 4 Gallons**
- **Jug B = 3 Gallons**

Neither jug has any measurement markings. There is an unlimited water supply available through a pump.

The following operations are allowed:

- Fill a jug completely from the pump.
- Empty a jug onto the ground.
- Pour water from one jug into the other until either the source jug becomes empty or the destination jug becomes full.

Objective: Obtain exactly 2 gallons of water in the 4-gallon jug.

1. Introduction

- The Water Jug Problem is one of the classic problems in Artificial Intelligence (AI) used to demonstrate state-space search and problem-solving techniques. It illustrates how an intelligent agent systematically explores different states to reach a desired goal while satisfying given constraints.
- Each state represents the amount of water in the two jugs, and every action changes the current state into a new state. AI search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), or State Space Search can be applied to determine a valid sequence of operations. Since the goal is to obtain exactly 2 gallons in the 4-gallon jug, the search continues until a goal state is reached.
- The Water Jug Problem is a good example of how AI models real-world decision-making by representing problems through states, operators, transitions, and goal conditions.

2. Problem Understanding

Two unmarked jugs with capacities of 4 gallons and 3 gallons are available. The objective is to measure exactly 2 gallons in the 4-gallon jug without using any additional measuring device. The only available operations are filling, emptying, and transferring water between the two jugs.

The problem is solved by exploring different states and selecting valid actions until the goal state is achieved.

3. Assumptions

The following assumptions are considered while solving the problem:

- An unlimited water supply is available.
- Water may be poured onto the ground at any time.
- Water may be transferred between the two jugs.
- Jugs have no measurement markings.
- The objective is to obtain exactly 2 gallons in the 4-gallon jug.

4. Problem Formulation

Initial State

Both jugs are empty.

[
(0,0)
]

where

- First value = Water in the **4-gallon jug**
- Second value = Water in the **3-gallon jug**

Goal State

[
(2,x)
]

where the 4-gallon jug contains exactly **2 gallons**, and the amount in the 3-gallon jug can be any value.

State Representation

A state is represented as:

[
(x,y)
]

where

- x = Water in 4-gallon jug
- y = Water in 3-gallon jug

Example:

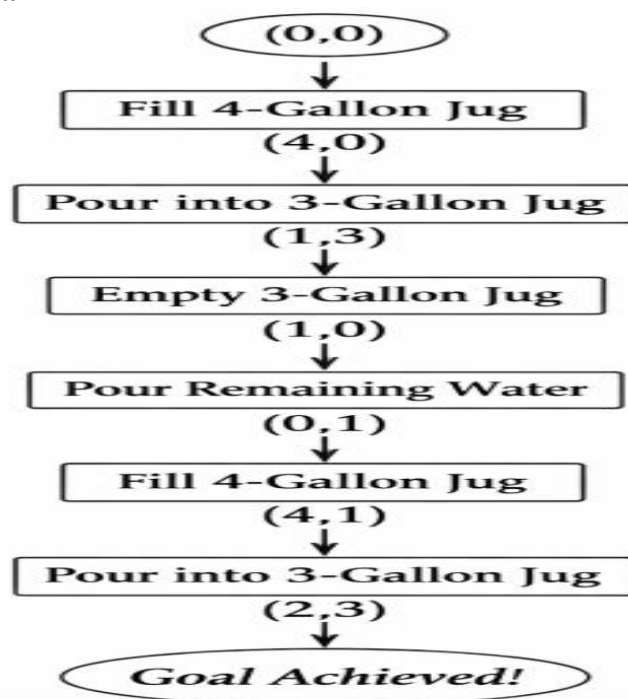
(4,0)

means

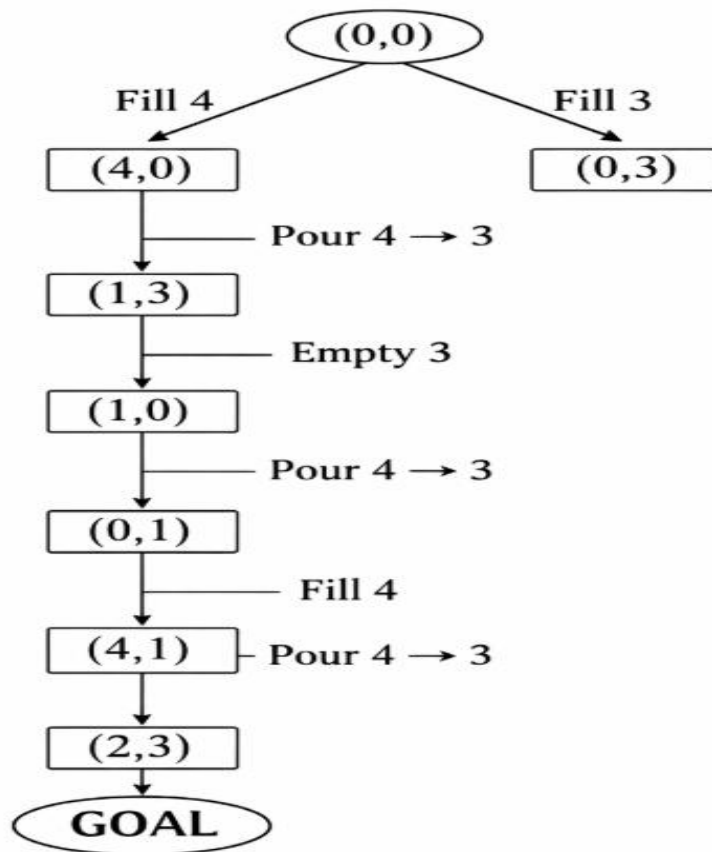
4-gallon jug is full

3-gallon jug is empty.

5. State Space Diagram



6. State Space Search Tree



7. Production Rules (Operators)

The intelligent agent can perform the following actions:

Rule Operation

- R1 Fill the 4-gallon jug
- R2 Fill the 3-gallon jug
- R3 Empty the 4-gallon jug
- R4 Empty the 3-gallon jug
- R5 Pour water from the 4-gallon jug into the 3-gallon jug
- R6 Pour water from the 3-gallon jug into the 4-gallon jug

These operators define all possible state transitions in the problem.

8. Step-by-Step Solution

Step 1

Initial State

[
 $(0,0)$
]

Both jugs are empty.

Step 2

Fill the 4-gallon jug.

Action:

Fill Jug A.

State

```
[  
(4,0)  
]
```

Step 3

Pour water from the 4-gallon jug into the 3-gallon jug until the smaller jug becomes full.

State

```
[  
(1,3)  
]
```

Explanation:

- 3 gallons transferred.
- 1 gallon remains in the 4-gallon jug.

Step 4

Empty the 3-gallon jug.

State

```
[  
(1,0)  
]
```

Step 5

Transfer the remaining 1 gallon from the 4-gallon jug into the 3-gallon jug.

State

```
[  
(0,1)  
]
```

Step 6

Fill the 4-gallon jug again.

State

```
[  
(4,1)  
]
```

Step 7

Pour water from the 4-gallon jug into the 3-gallon jug.

The 3-gallon jug already contains 1 gallon, so it requires only 2 additional gallons to become full.

After transferring 2 gallons,

State

```
[  
(2,3)  
]
```

Goal State Achieved

The 4-gallon jug now contains exactly **2 gallons**.

9. State Transition Table

Step	Action	State (4-Gallon, 3-Gallon)
1	Initial	(0,0)
2	Fill 4-Gallon Jug	(4,0)
3	Pour 4 $\rightarrow$ 3	(1,3)
4	Empty 3-Gallon Jug	(1,0)
5	Pour 4 $\rightarrow$ 3	(0,1)
6	Fill 4-Gallon Jug	(4,1)
7	Pour 4 $\rightarrow$ 3	(2,3)

10. Algorithm (State Space Search)

1. Start with both jugs empty.
2. Fill the 4-gallon jug.
3. Pour water into the 3-gallon jug until it is full.
4. Empty the 3-gallon jug.
5. Transfer the remaining water from the 4-gallon jug to the 3-gallon jug.
6. Refill the 4-gallon jug.
7. Pour water from the 4-gallon jug into the 3-gallon jug until it is full.
8. Stop when the 4-gallon jug contains exactly 2 gallons.

11. Advantages of State Space Search

- Systematically explores all possible states.
- Guarantees a valid solution when one exists.
- Avoids illegal operations.
- Suitable for AI problem-solving applications.
- Can be implemented using BFS or DFS algorithms.
- Forms the basis for many intelligent planning systems.

12. Applications

The Water Jug Problem demonstrates concepts used in many real-world AI applications, including:

- Robotic motion planning
- Automated resource allocation
- Industrial process control
- Intelligent planning systems
- Game solving
- Route optimization
- Constraint satisfaction problems

13. Justification

The Water Jug Problem is naturally modeled as a state-space search problem, where each state represents the current amount of water in both jugs. The available operations (fill, empty, and pour) act as state-transition operators. By systematically exploring these states, the search algorithm guarantees that the goal state **(2,3)** is reached without violating any constraints. This approach is efficient because it considers only valid state transitions and avoids impossible configurations. Consequently, state-space search provides a complete and logical solution to the problem.

14. Conclusion

The Water Jug Problem demonstrates how Artificial Intelligence solves real-world problems using state-space search techniques. By representing each water configuration as a state and applying valid operations as transitions, the intelligent agent systematically searches for a solution. Starting from the initial state (0,0), the algorithm successfully reaches the goal state (2,3), where the 4-gallon jug contains exactly 2 gallons. This example highlights the effectiveness of AI search strategies in solving planning and decision-making problems and forms the foundation for more advanced applications in robotics, automation, and intelligent systems.

Question 2: Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- vi. **What are the percept's for this agent?**
- vii. **Characterize the operating environment.**
- viii. **What are the actions the agent can take?**
- ix. **How can one evaluate the performance of the agent?**
- x. **What sort of agent architecture do you think is most suitable for this agent? Why?**

Mars Rover – Artificial Intelligence Agent for Exploring and Analyzing Samples on Mars

Introduction

The exploration of Mars is one of the most significant achievements in space science and Artificial Intelligence. Since human exploration of Mars is currently limited by extreme environmental conditions, space agencies such as NASA and the European Space Agency (ESA) have developed autonomous Mars Rovers capable of navigating the Martian surface, collecting geological samples, analyzing rocks and soil, detecting water and minerals, and transmitting scientific data back to Earth.

A Mars Rover functions as an Intelligent Autonomous Agent because it can perceive its surroundings using sensors, reason about its environment, make decisions independently, and execute actions to accomplish predefined scientific objectives. Unlike robots operating in controlled environments, a Mars Rover must work in an unknown, partially observable, dynamic, and hazardous environment where communication delays with Earth make continuous human control impractical.

Artificial Intelligence enables the rover to perform autonomous navigation, obstacle avoidance, route planning, scientific sample collection, and adaptive decision-making. This significantly improves mission success while reducing the need for constant human intervention.

Problem Understanding

A Mars Rover is deployed on the Martian surface with the primary objective of exploring unknown regions, analyzing rocks and soil, searching for evidence of water and life, and transmitting scientific observations to mission control on Earth.

Since the Martian environment is unpredictable, the rover must continuously observe its surroundings, identify obstacles, determine safe routes, and perform scientific experiments independently. The rover behaves as an intelligent agent by sensing, reasoning, planning, learning from observations, and taking appropriate actions to accomplish its mission.

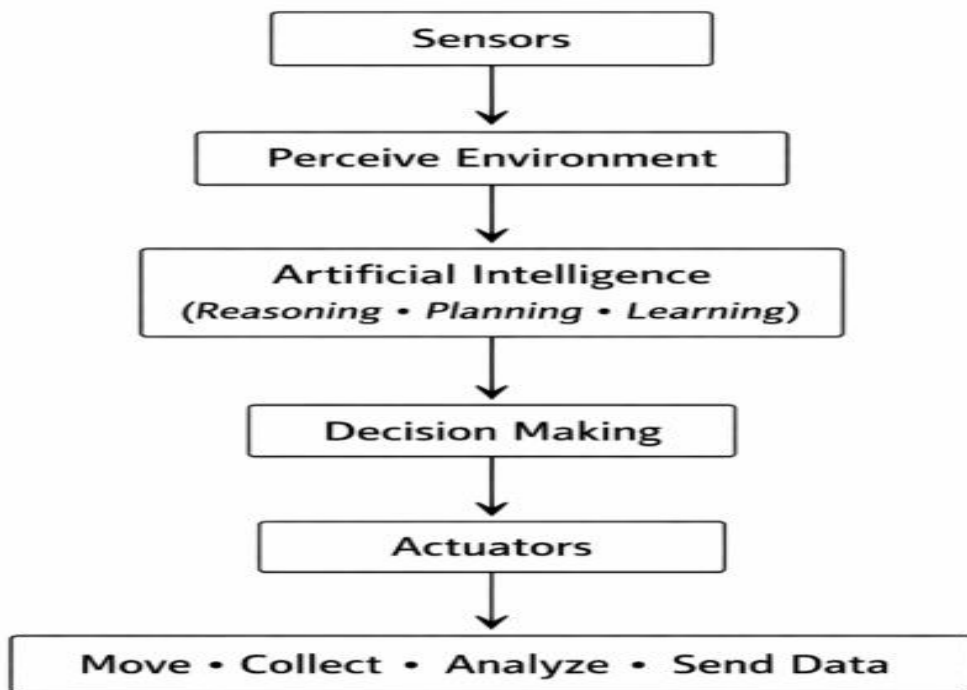
Objectives of the Mars Rover

The primary objectives of the Mars Rover are:

- Explore unknown regions of Mars.
- Collect soil and rock samples.
- Analyze geological and chemical composition.
- Search for water, minerals, and signs of ancient life.
- Avoid hazardous terrain and obstacles.
- Capture high-resolution images.
- Transmit scientific data to Earth safely and efficiently.

Architecture of the Mars Rover as an Intelligent Agent

Mars Rover Intelligent Agent



(i) What are the Percepts for this Agent?

Percepts

Percepts are the inputs received by the Mars Rover through its sensors. These inputs allow the rover to understand its surroundings and make intelligent decisions.

Major Percepts Sensor	Information Collected
Navigation Camera	Terrain, rocks, obstacles, slopes
Stereo Camera	Distance and depth information
Spectrometer	Mineral and chemical composition
Temperature Sensor	Surface temperature
Atmospheric Sensor	Pressure, humidity, wind conditions
Radiation Sensor	Radiation levels
Soil Sensor	Soil characteristics
GPS/Localization System	Current position (estimated using onboard navigation)

Analysis

These percepts provide continuous environmental information that helps the rover navigate safely, identify scientifically important samples, and avoid dangerous regions. Without accurate percepts, the rover cannot make intelligent decisions.

(ii) Characterize the Operating Environment

The Mars Rover operates in a highly challenging environment that can be characterized using the standard AI environment properties.

Environment Property	Description
----------------------	-------------

Observability	Partially Observable – The rover cannot observe the complete environment simultaneously.
Determinism	Stochastic – Outcomes may vary because of dust storms, terrain changes, and wheel slippage.
Agents	Single Agent – Only one autonomous rover performs the mission.
Dynamics	Dynamic – Environmental conditions change continuously.
State Space	Sequential – Current decisions influence future actions.
Knowledge	Initially Unknown – The rover gradually builds knowledge while exploring.
Environment Type	Continuous – Movement and sensing occur continuously over time.
Environment Property	Description

Analysis

Because the rover has incomplete information and environmental conditions constantly change, it must continuously update its knowledge base and adapt its navigation strategy.

(iii) What Actions Can the Agent Perform?

The Mars Rover performs several intelligent actions to accomplish its exploration mission.

Navigation Actions

- Move Forward
- Move Backward
- Turn Left
- Turn Right
- Climb Slopes
- Avoid Obstacles
- Stop Safely
-

Scientific Actions

- Capture Images
- Drill Rocks
- Collect Soil Samples
- Analyze Rock Composition
- Detect Water Molecules
- Measure Atmospheric Conditions

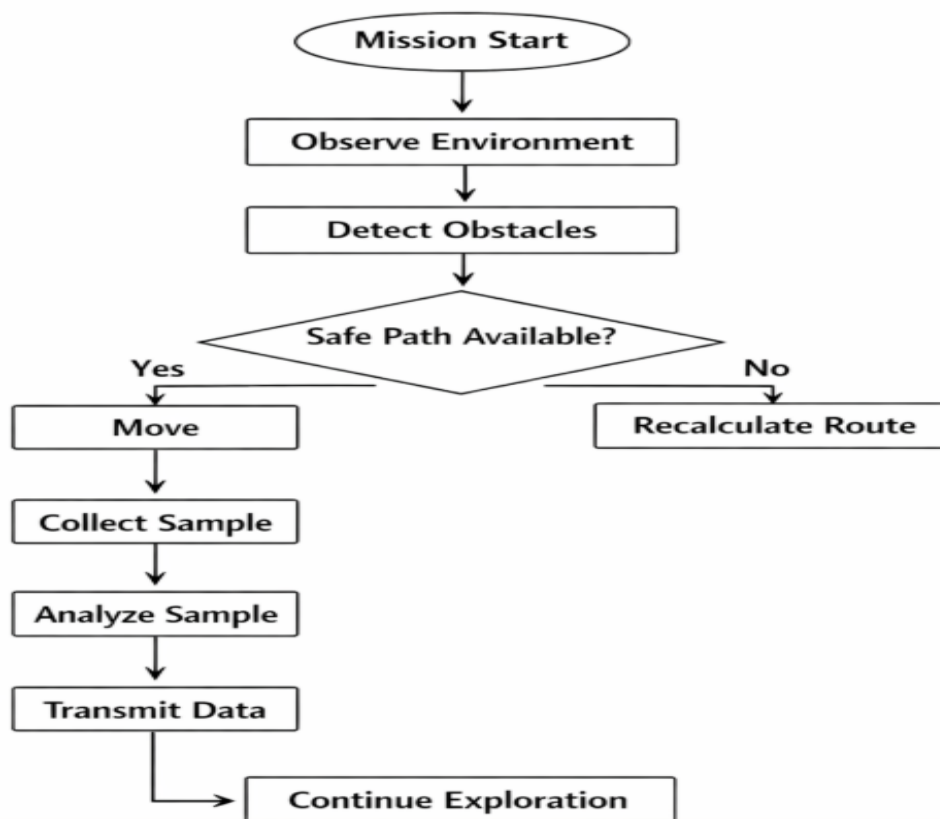
Communication Actions

- Store Scientific Data
- Compress Images
- Transmit Information to Earth
- Receive Updated Mission Commands

Self-Monitoring Actions

- Monitor Battery Level
- Recharge Using Solar Panels
- Diagnose Hardware Status

Mars Rover Action Flow



(iv) How Can the Performance of the Agent Be Evaluated?

The effectiveness of the Mars Rover is measured using **performance measures**, which indicate how successfully the mission objectives are achieved.

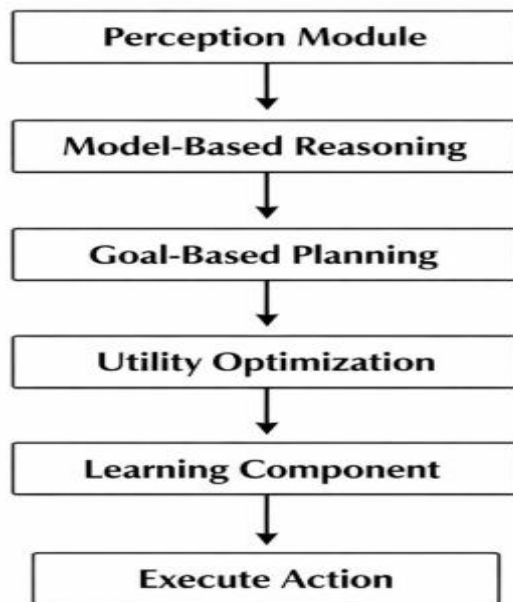
Performance Measure	Evaluation Criteria
Navigation Accuracy	Ability to reach target locations safely
Obstacle Avoidance	Number of collisions avoided
Sample Collection Efficiency	Number and quality of samples collected
Scientific Accuracy	Correct identification of minerals and compounds
Energy Efficiency	Minimum battery consumption
Communication Reliability	Successful transmission of scientific data
Mission Completion Time	Time required to complete exploration tasks
System Reliability	Ability to operate without failures

Analysis

A successful Mars Rover should maximize scientific discoveries while minimizing energy consumption, travel time, and operational risks. (v) **Which Agent Architecture is Most Mars Rover is the Hybrid Intelligent Agent Suitable? Why?** Hybrid Intelligent Agent

Components of Hybrid Architecture

Hybrid Intelligent Agent



Why Hybrid Architecture?

The Mars Rover requires multiple AI capabilities simultaneously.

- **Model-Based Component** maintains an internal representation of the environment.
- **Goal-Based Component** plans actions to achieve mission objectives.
- **Utility-Based Component** selects actions that maximize scientific benefit while minimizing risk.
- **Learning Component** improves navigation and decision-making based on previous experiences.

Therefore, Hybrid Architecture combines the strengths of multiple AI agent types and provides greater autonomy than any single architecture.

Advantages of AI in Mars Rover Exploration

- Autonomous navigation in unknown environments.
- Intelligent obstacle avoidance.
- Real-time scientific analysis.
- Efficient resource utilization.
- Reduced communication dependency.
- Adaptive learning from new observations.
- Increased mission reliability.
- Enhanced exploration efficiency.

Real-World Applications

The AI technologies used in Mars Rovers are also applied in:

- Autonomous vehicles
- Search-and-rescue robots
- Underwater exploration robots
- Space exploration missions

- Military surveillance systems
- Agricultural robots
- Industrial inspection robots
- Planetary exploration missions

Justification

The Mars Rover operates in a partially observable, dynamic, and uncertain environment, where continuous human supervision is not feasible due to communication delays between Mars and Earth. A Hybrid Intelligent Agent Architecture is the most suitable choice because it integrates perception, reasoning, planning, utility optimization, and learning into a single system. This enables the rover to autonomously navigate, avoid obstacles, collect scientifically valuable samples, conserve energy, and adapt to unexpected environmental conditions. By combining these AI capabilities, the rover can maximize mission success while minimizing operational risks, making the hybrid approach highly effective for extraterrestrial exploration.

Conclusion

The Mars Rover is an excellent example of an Artificial Intelligence–based autonomous intelligent agent capable of performing complex exploration tasks in one of the most challenging environments known. Through advanced sensing, reasoning, planning, and learning, it can independently navigate the Martian surface, analyze geological samples, avoid hazards, and communicate valuable scientific information back to Earth. The use of a Hybrid Intelligent Agent Architecture ensures that the rover operates rationally, efficiently, and adaptively in a partially observable and dynamic environment. Consequently, Artificial Intelligence plays a vital role in expanding human knowledge of Mars and advancing future space exploration missions..

Question 3: Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column. Explore the Search Space Using Search Strategies and Formulate the Problem Components for the 8-Queens Problem

Introduction

- The 8-Queens Problem is one of the classical problems in Artificial Intelligence (AI) and Constraint Satisfaction Problems (CSPs). It demonstrates how intelligent search algorithms systematically explore a large search space to find a valid solution while satisfying a set of constraints.
- The objective of the problem is to place eight queens on an 8×8 chessboard in such a way that no two queens attack each other. Since a queen can move horizontally, vertically, and diagonally, the placement must ensure that no two queens share the same row, column, or diagonal.
- The 8-Queens Problem is widely used to study state-space search, backtracking search, heuristic search, local search, and constraint satisfaction techniques. It serves as a benchmark problem for evaluating AI search algorithms because the search space is very large, requiring intelligent pruning and efficient exploration to obtain a valid solution.

Problem Understanding

The problem consists of placing eight queens on a standard 8×8 chessboard while satisfying all attack constraints.

The challenge is that each queen attacks:

- Every square in the same row

- Every square in the same column
- Every square along both diagonals

The goal is to find a configuration where all queens coexist without attacking one another.

Objective

The objective is to determine a valid arrangement of eight queens using an appropriate AI search strategy such that:

- Every row contains exactly one queen.
- Every column contains exactly one queen.
- No two queens share the same diagonal.

Problem Formulation

State

A state represents the current arrangement of queens placed on the chessboard.

Example

Q - - - - -

- - Q - - - -

- - - - Q - - -

...

Each state may be a **partial solution** or a **complete solution**.

Initial State

An empty chessboard.

- - - - -

- - - - -

- - - - -

- - - - -

- - - - -

- - - - -

- - - - -

- - - - -

Goal State

A complete chessboard containing **8 queens** satisfying all constraints.

Example

Q - - - - -

- - - - Q - - -

- - - - - Q

- - - - - Q - -

```

-- Q -----
----- Q -
- Q -----
--- Q ----

```

Problem Components

Component	Description
Initial State	Empty chessboard
State Space	All possible queen placements
Operators	Place one queen in a safe position
Goal Test	Eight queens placed safely
Path Cost	Usually equal for every move

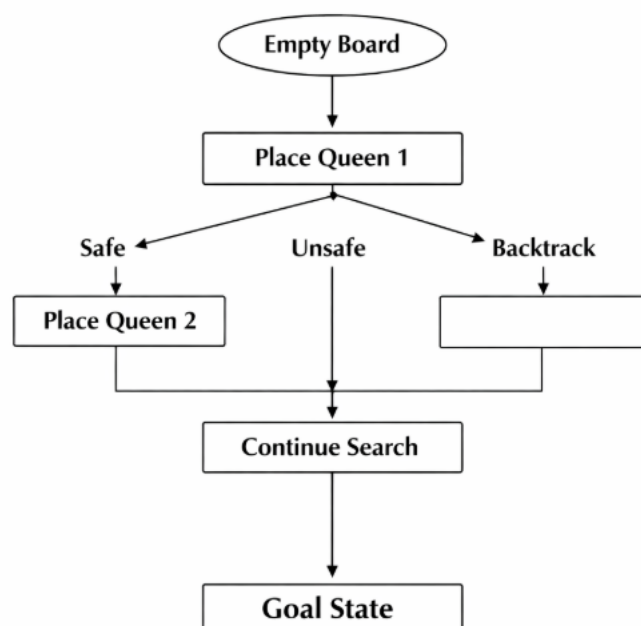
Constraint Analysis

The 8-Queens Problem is governed by three major constraints.

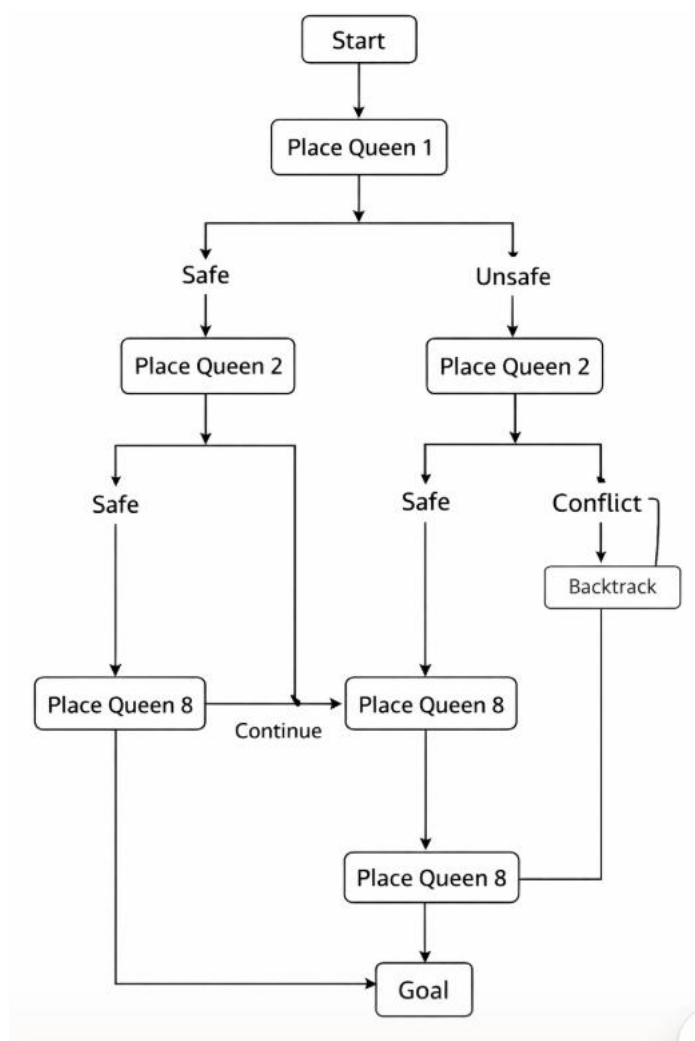
Constraint	Explanation	Effect
Row Constraint	One queen per row	Eliminates horizontal attacks
Column Constraint	One queen per column	Eliminates vertical attacks
Diagonal Constraint	No queens share diagonals	Eliminates diagonal attacks

These constraints significantly reduce the search space by eliminating invalid placements during the search process.

Search Space Representation



State Space Tree



Search Strategy

The most suitable search strategy for solving the 8-Queens Problem is the **Backtracking Search Algorithm**.

Why Backtracking?

Backtracking constructs the solution incrementally.

Whenever a newly placed queen violates any constraint, the algorithm immediately removes that queen and tries another position.

Thus, unnecessary exploration is avoided, making the search more efficient than brute-force search.

Backtracking Algorithm

Step 1

Start with an empty chessboard.

Step 2

Place the first queen in the first row.

Step 3

Move to the next row.

Step 4

Check

- Column safety
- Left diagonal safety
- Right diagonal safety

Step 5

If safe,
Place queen.
Else,
Try another column.

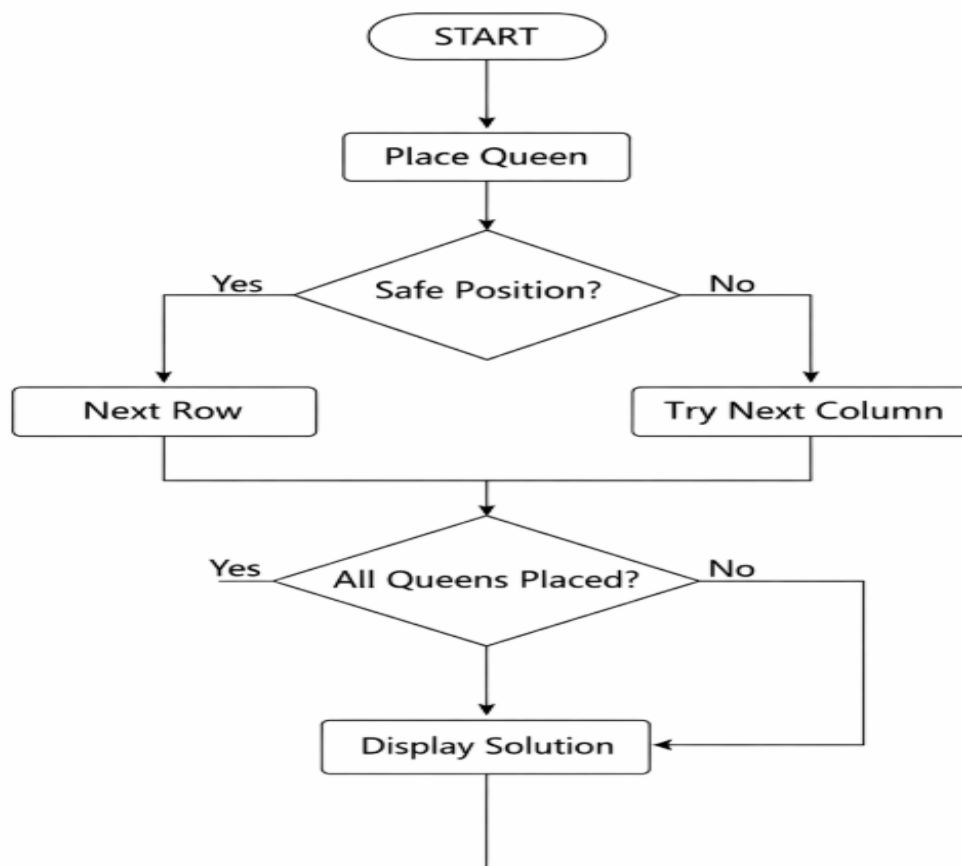
Step 6

If no safe column exists,
Backtrack to the previous row.

Step 7

Repeat until all eight queens are placed.

Flowchart



Example Search Process

Step	Action	Result
1	Place Queen in Row 1	Safe
2	Place Queen in Row 2	Safe
3	Place Queen in Row 3	Conflict
4	Backtrack	Previous Queen Removed
5	Choose Another Column	Safe
6	Continue Search	Valid Configuration

Application of Artificial Intelligence Concepts

Intelligent Search

The algorithm intelligently explores only valid positions instead of checking every possible arrangement.

Constraint Satisfaction Problem

Each queen placement must satisfy all three constraints.

Violation of even one constraint results in rejection of the state.

State Space Search

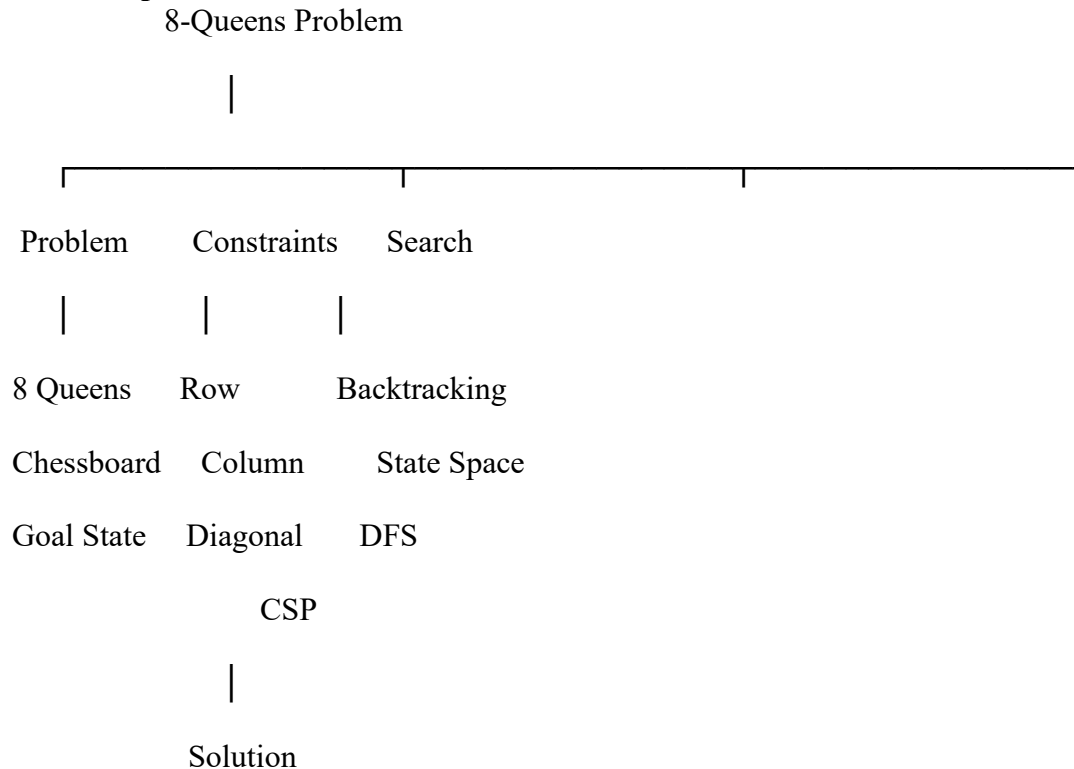
Each chessboard configuration represents one state.

The algorithm explores the state space systematically until reaching the goal state.

Backtracking

Backtracking prevents unnecessary computations by abandoning invalid partial solutions immediately.

Mind Map



Advantages of Backtracking Search

- Efficient exploration of the search space.
- Eliminates invalid solutions early.
- Guarantees a valid solution if one exists.
- Reduces computational complexity compared with brute-force search.
- Widely applicable to many AI optimization problems.

Applications

The concepts used in the 8-Queens Problem are applied in:

- University timetable scheduling
- Hospital staff scheduling
- Airline crew scheduling
- Register allocation in compilers
- Robot path planning
- Sudoku solving
- Map coloring
- Resource allocation systems

Justification

The Backtracking Search Algorithm is the most appropriate technique for solving the 8-Queens Problem because it treats the problem as a Constraint Satisfaction Problem (CSP). Rather than exploring every possible arrangement, the algorithm checks the validity of each queen placement as soon as it is made. If a conflict occurs in any row, column, or diagonal, it immediately backtracks and explores an alternative position. This strategy significantly reduces the search space, avoids unnecessary computations, and guarantees a correct solution whenever one exists. Consequently, Backtracking Search provides an efficient and systematic approach for solving the 8-Queens Problem.

Conclusion

The 8-Queens Problem is a fundamental Artificial Intelligence problem that demonstrates the application of state-space search, constraint satisfaction, and backtracking algorithms. By representing each queen placement as a state and enforcing row, column, and diagonal constraints, the search algorithm systematically explores valid configurations while eliminating invalid ones through backtracking. This intelligent search process efficiently identifies a solution in which all eight queens coexist without attacking one another. Beyond chess, the same principles are extensively applied in scheduling, planning, optimization, robotics, and resource allocation, making the 8-Queens Problem a classic and practical example of AI-based problem solving.

Question 4: A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problemA Customer Wants to Travel Using the OLA Cab Booking Mobile Application. Identify the Type of Problem-Solving Agent Used and Write the Pseudocode.

1. Introduction

Artificial Intelligence (AI) plays a vital role in modern transportation systems by providing intelligent decision-making, route optimization, and personalized services. Ride-hailing applications such as OLA, Uber, and Lyft utilize AI to connect passengers with nearby drivers while optimizing travel time, cost, and customer satisfaction.

When a customer requests a cab, the system must analyze multiple factors such as the customer's location, destination, available cab types, traffic conditions, estimated fare, driver availability, and travel time. Based on these inputs, the system intelligently recommends the most suitable cab option. This decision-making process is an example of an AI Problem-Solving Agent, which uses available information to achieve a specific goal efficiently.

The OLA Cab Booking System demonstrates how Artificial Intelligence can automate transportation services by enabling users to choose from different cab categories such as Mini, Micro, Sedan, Prime, Shared, and SUV, based on their comfort, budget, and travel requirements.

2. Problem Understanding

A customer intends to travel from one location to another using the OLA mobile application. The customer enters the pickup location and destination and selects a preferred cab category based on comfort, budget, and travel needs.

The system identifies nearby drivers, calculates the estimated fare and travel time, suggests the available cab options,

minimizing waiting time, travel cost, and inconvenience.

3. Objective

The objective of the OLA Cab Booking System is to:

- Identify the customer's travel request.
- Recommend the best available cab.
- Minimize waiting time.
- Optimize travel cost and route.
- Ensure customer satisfaction.
- Complete the booking successfully.

4. Type of AI Problem-Solving Agent

Goal-Based Agent

The **Goal-Based Agent** is the most appropriate AI agent for the OLA Cab Booking System.

A Goal-Based Agent selects actions that help achieve a specific goal. In this case, the goal is to transport the customer safely from the pickup location to the destination using the most suitable cab.

Unlike a simple reflex agent, a Goal-Based Agent considers future outcomes before making decisions. It evaluates multiple alternatives, such as cab availability, fare, estimated arrival time, traffic conditions, and customer preferences, before selecting the best option.

5. Why Goal-Based Agent?

The Goal-Based Agent is suitable because it:

- Identifies the customer's destination.
- Considers multiple cab options.
- Evaluates travel cost and estimated arrival time.
- Selects the optimal cab based on customer preferences.
- Achieves the predefined goal efficiently.

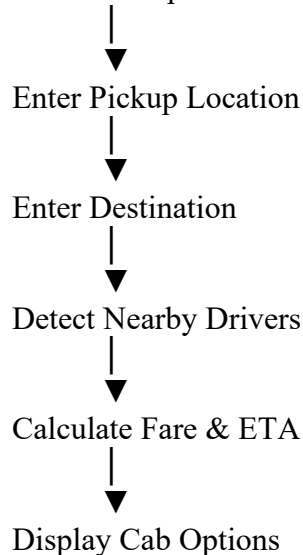
6. AI Components of the OLA Booking System

AI Component Description

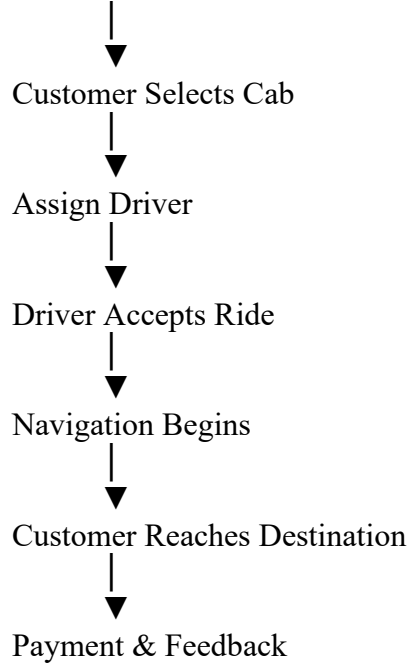
Agent	OLA Booking System
Environment	Roads, traffic, customers, drivers
Sensors	GPS, customer input, traffic data, driver status
Actuators	Mobile notifications, booking confirmation, route guidance
Goal	Successfully transport the customer to the destination

7. Working of the OLA Booking System

Customer Opens OLA App



(Mini, Micro, Sedan, Prime, Shared)



8. Problem Formulation

Initial State

Customer opens the OLA application.

\

State Space

All available cab types and nearby drivers.

Operators

- Enter pickup location
- Enter destination
- Search available cabs
- Select cab type
- Assign driver
- Navigate to destination
- Complete payment

Goal State

Customer successfully reaches the destination.

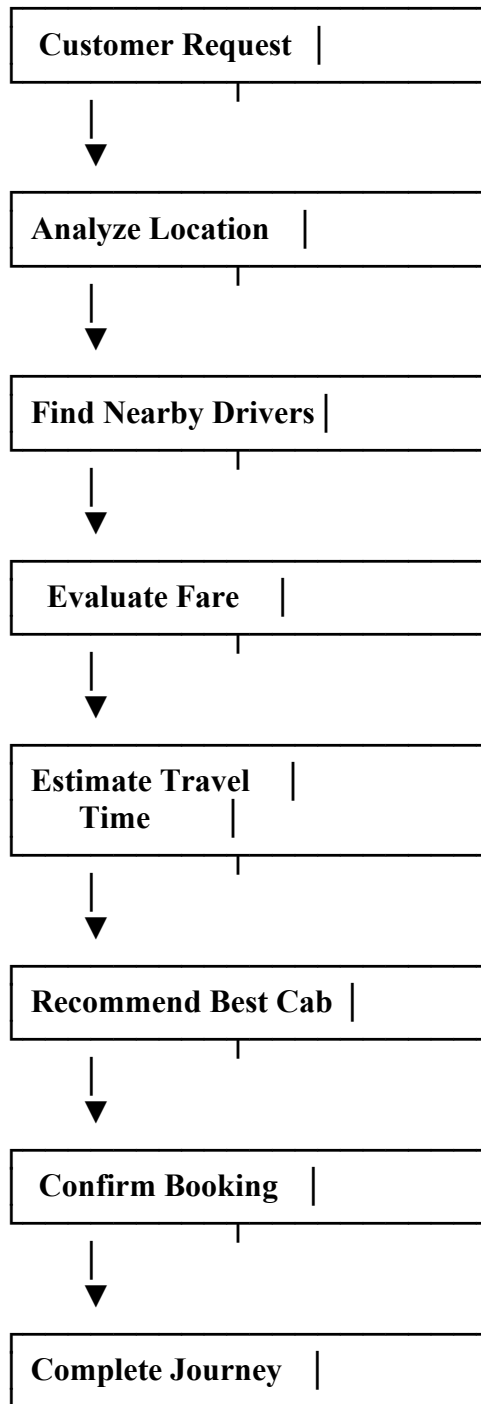
Path Cost

The path cost includes:

- Estimated fare
- Waiting time
- Travel distance
- Traffic conditions
- Travel duration

The AI system aims to minimize the total path cost.

9. Decision-Making Process



10. Pseudocode

BEGIN

Input Pickup_Location

Input Destination

Search Available_Cabs

IF No Cab Available THEN

 Display "No Cabs Available"

 EXIT

ENDIF

Calculate Estimated_Fare

Calculate Estimated_Time

Display Available Cab Types

Customer Selects Cab

Assign Nearest Driver

IF Driver Accepts THEN

 Confirm Booking

 Navigate to Destination

 Complete Trip

 Generate Bill

 Collect Customer Feedback

ELSE

 Assign Another Driver

ENDIF

END

11. Intelligent Agent Architecture

Goal-Based Agent

Sensors

GPS | Customer Input | Traffic

|



Decision Making System

|

Route + Fare + Driver Selection

|



Actuators

Booking | Navigation | Notifications

12. Performance Measures

The performance of the OLA Booking System can be evaluated using:

Performance Measure Description

Waiting Time Time taken to assign a driver

Fare Accuracy Correct fare estimation

Customer Satisfaction User ratings and reviews

Performance Measure Description

Driver Availability	Successful driver assignment
Route Optimization	Shortest and fastest route
Booking Success Rate	Number of successful bookings
Travel Time	Total journey duration

13. Advantages of Goal-Based Agent

- Makes intelligent decisions.
- Selects the most suitable cab.
- Optimizes travel routes.
- Reduces customer waiting time.
- Improves customer satisfaction.
- Efficiently handles dynamic traffic conditions.
- Supports real-time decision making.

14. Real-World Applications

The Goal-Based Agent approach is widely used in:

- OLA
- Uber
- Rapido
- Swiggy Delivery
- Zomato Delivery
- Amazon Logistics
- Google Maps Navigation
- Autonomous Vehicles

15. Justification

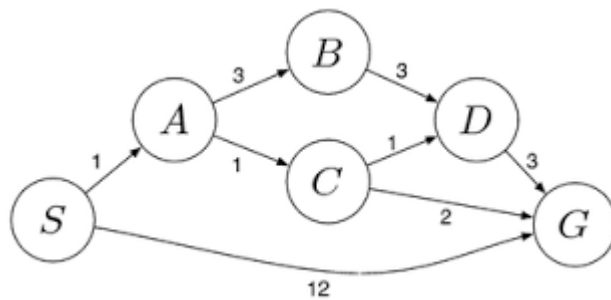
The **Goal-Based Agent** is the most suitable architecture for the OLA Cab Booking System because it focuses on achieving a clearly defined objective: transporting the customer safely and efficiently to the destination. It evaluates multiple factors, including driver availability, estimated fare, traffic conditions, route optimization, and customer preferences, before selecting the most appropriate cab. Unlike a simple reflex agent, it plans ahead and chooses actions that maximize service quality while minimizing travel time and cost. This intelligent decision-making process enhances customer satisfaction and ensures efficient transportation.

16. Conclusion

The OLA Cab Booking System is an excellent example of an AI-based **Goal-Based Agent** that intelligently analyzes customer requirements, evaluates available cab options, and recommends the most suitable service. By integrating GPS, traffic analysis, route optimization, and real-time driver availability, the system provides safe, efficient, and cost-effective transportation. Its ability to make informed decisions based on predefined goals demonstrates the practical application of Artificial Intelligence in modern transportation systems. The Goal-Based Agent architecture ensures high reliability, improved user experience, and optimal resource utilization, making it the ideal solution for intelligent ride-hailing applications.

Question 5: A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Finding the Least-Cost Delivery Route Using Uniform Cost Search (UCS)

1. Introduction

Artificial Intelligence provides intelligent search algorithms for solving real-world optimization problems involving transportation, logistics, and route planning. One of the most effective uninformed search techniques is the Uniform Cost Search (UCS) Algorithm, which expands the node having the lowest cumulative path cost rather than the shortest distance.

In logistics and supply chain management, transportation cost directly affects business profitability. Therefore, selecting the least-cost delivery route is essential for minimizing fuel consumption, reducing delivery time, and improving operational efficiency.

In this problem, a logistics company must transport packages from the Starting Warehouse (S) to the Destination Warehouse (G). Every road between warehouses has an associated travel cost. The objective is to determine the minimum-cost delivery path using the Uniform Cost Search (UCS) algorithm.

2. Problem Understanding

The logistics company operates several interconnected warehouses represented as a weighted graph.

Each edge represents the transportation cost between two warehouses.

The objective is to identify the **least-cost path** from

Start Warehouse (S)

to

Goal Warehouse (G)

using Uniform Cost Search.

Unlike Breadth First Search, UCS considers the total accumulated travel cost before expanding a node.

3. Problem Representation

Initial State

Start Warehouse

[
S
]

Goal State

Destination Warehouse

[
G
]

State Space

Each warehouse represents one state.

[
 {S,A,B,C,D,G}
]

Operators

Move from one warehouse to another through connected routes.

Path Cost

Sum of all edge costs from

Start

to

Goal.

4. Weighted Graph

From the figure,

Route Cost

$S \rightarrow A$ 1

$S \rightarrow G$ 12

$A \rightarrow B$ 3

$A \rightarrow C$ 1

$B \rightarrow D$ 3

$C \rightarrow D$ 1

$C \rightarrow G$ 2

$D \rightarrow G$ 3

Every road has a travel cost	UCS minimizes total cumulative cost instead of number of edges.
Multiple routes exist	The algorithm evaluates all possible paths before selecting the cheapest.
Positive edge weights	UCS guarantees the optimal solution.
Goal is minimum transportation cost	Lowest-cost path is always expanded first.
Every road has a travel cost	UCS minimizes total cumulative cost instead of number of edges.

6. Uniform Cost Search Algorithm

Algorithm

1. Insert Start node into Priority Queue.
2. Remove the node having minimum cumulative cost.
3. Expand neighbouring nodes.
4. Calculate cumulative path cost.
5. Insert newly generated nodes into Priority Queue.
6. Repeat until Goal node is removed.

7. Search Tree

S(0)

/ \

A(1)

G(12)

/ \

B(4) C(2)

/ \

D(3) G(4)

|

G(6)

Numbers represent cumulative costs.

8. Step-by-Step UCS Execution

Step 1

Priority Queue

S(0)

Expand

S

Neighbours

A = 1

G = 12

Queue

A(1), G(12)

Step 2

Expand

A(1)

Neighbours

B

Cost

$1+3 = 4$

C

Cost

$1+1 = 2$

Queue

C(2)

B(4)

G(12)

Step 3

Expand

C(2)

Neighbours

D

Cost

$2+1 = 3$

G

Cost

$2+2 = 4$

Queue

D(3)

B(4)

G(4)

G(12)

Step 4

Expand

D(3)

Neighbour

G

Cost

$3+3 = 6$

Queue

B(4)

G(4)

G(6)

G(12)

Step 5

Expand

B(4)

Neighbour

D

Cost

$4+3 = 7$

Queue

G(4)

G(6)

D(7)

G(12)

Step 6

Expand

G(4)

Goal reached.

Search Stops.

9. Priority Queue Table

Step Expanded Node Priority Queue After Expansion

1	S(0)	A(1), G(12)
2	A(1)	C(2), B(4), G(12)
3	C(2)	D(3), B(4), G(4), G(12)
4	D(3)	B(4), G(4), G(6), G(12)
5	B(4)	G(4), G(6), D(7), G(12)

Step Expanded Node Priority Queue After Expansion

6 G(4)

Goal Found

10. Optimal Path Calculation

Selected Route

S

↓

A

↓

C

↓

G

Total Cost

[

1+1+2=4

]

11. Final Solution

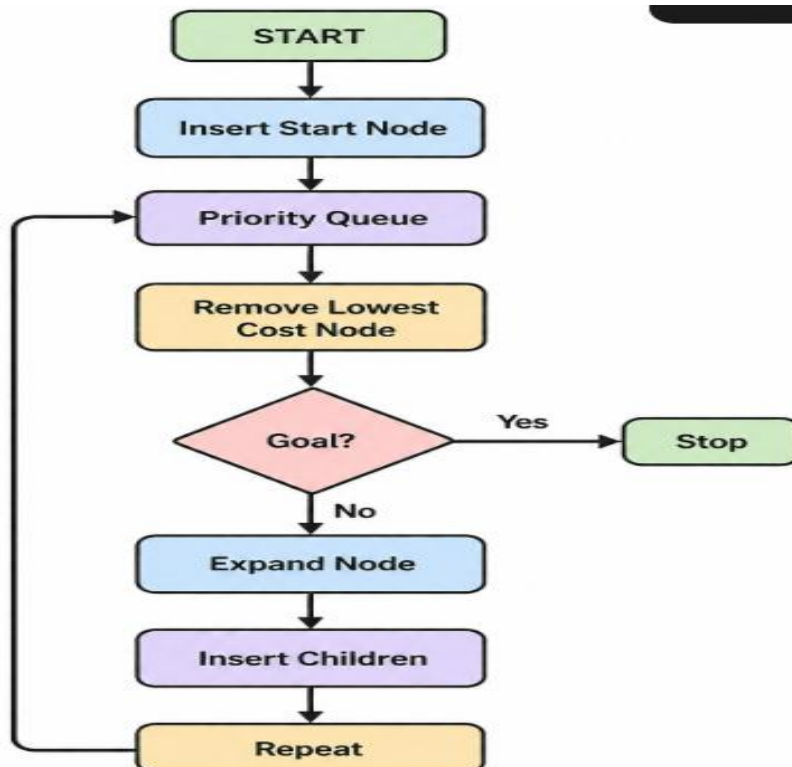
Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Minimum Cost

4

12. UCS Flowchart



13. Advantages of Uniform Cost Search

- Always finds the optimal solution.
- Suitable for weighted graphs.
- Expands lowest-cost node first.
- Guarantees minimum transportation cost.
- Efficient for logistics planning.
- Supports intelligent route optimization.

14. Real-Time Applications

Uniform Cost Search is widely used in:

- Logistics and Supply Chain Management
- GPS Navigation Systems
- Google Maps Route Planning
- Autonomous Vehicle Navigation
- Robot Path Planning
- Airline Route Optimization
- Delivery Services (Amazon, DHL, FedEx)
- Smart Transportation Systems

15. Justification

Uniform Cost Search is the most suitable algorithm for this logistics problem because the objective is to minimize the total transportation cost rather than simply minimizing the number of routes. UCS expands nodes based on the lowest cumulative path cost, ensuring that the first time the goal node is selected, the path found is optimal. In this problem, although there is a direct path from S to G with a cost of 12, UCS correctly identifies the cheaper route $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of 4. This demonstrates the efficiency of UCS in solving weighted graph problems where route costs vary.

16. Conclusion

The **Uniform Cost Search (UCS)** algorithm successfully determines the least-cost delivery route in the logistics network by expanding nodes according to their cumulative travel costs. Through systematic exploration of the weighted graph, UCS identifies the optimal path $S \rightarrow A \rightarrow C \rightarrow G$ with a minimum total cost of **4**, which is significantly more efficient than the direct route from S to G. This example highlights the importance of AI search algorithms in real-world logistics, transportation, and route optimization, where minimizing operational cost is a critical objective.